Experiment 4: Generate Contextual Embeddings Using a Pre-trained Transformer Model
Aim:
To generate contextual embeddings from text using a pre-trained transformer model.

Algorithm:

1. Start the program.
2. Import the required PyTorch library.
3. Define the input text.
4. Convert the input text into token IDs using the BERT tokenizer.
5. Convert the token IDs into PyTorch tensors.
6. Set the BERT model to evaluation mode.
7. Disable gradient calculations using `torch.no_grad()`.
8. Pass the input tensors to the pre-trained BERT model.
9. Extract the contextual embeddings from `last_hidden_state`.
10. Display the embedding dimensions and selected embedding values.
11. Stop the program

Code:

```
import torch

text = "Artificial intelligence is changing the world."

# Convert text into model inputs

inputs = tokenizer(

    text,

    return_tensors="pt"

)

# Generate embeddings without calculating gradients

model.eval()

with torch.no_grad():

    outputs = model(**inputs)

# Contextual embeddings for every token

embeddings = outputs.last_hidden_state

print("Input text:", text)
```

print("Input token IDs:", inputs["input_ids"])

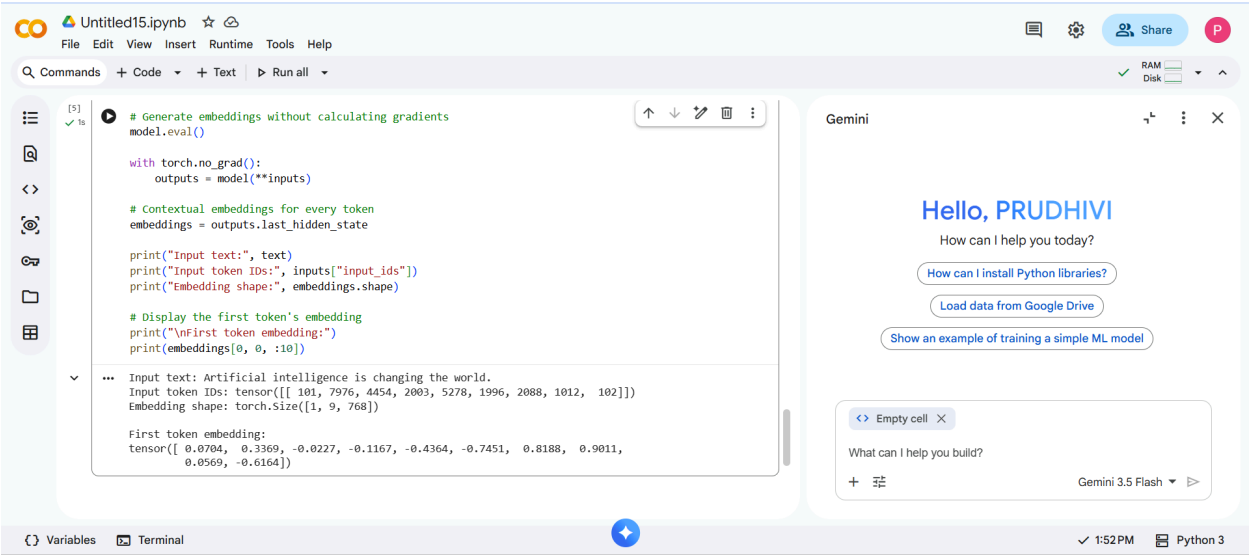
print("Embedding shape:", embeddings.shape)

Display the first token's embedding

print("\nFirst token embedding:")

print(embeddings[0, 0, :10])

Screenshot:



Result:

BERT generated numerical contextual embeddings for the input tokens, representing their learned contextual information.